CSA4704 – DEEP LEARNING

ASSIGNMENT 1: CO1 – MATHEMATICAL FOUNDATIONS OF DEEP LEARNING

Assessment Title: Analyse the Mathematical and Machine Learning Foundations of a Crop-Disease Screening System

Scenario: AgriVision Crop-Disease Screening

Student Name: Jeya Harshini R
Register Number: 192424051

ABSTRACT

This assignment analyses the mathematical and machine learning foundations of AgriVision, a proposed crop-disease screening system that classifies plant leaf images. The study uses the PlantVillage image dataset, focusing on three tomato leaf classes: Healthy, Early Blight and Late Blight. A total of 4,500 images were selected from a larger dataset containing 54,304 images and 38 crop-disease classes.

The images were resized to 64 × 64 pixels, converted to grayscale, normalized and represented as 4,096-dimensional feature vectors. Principal Component Analysis (PCA) was used to investigate feature redundancy and reduce dimensionality. Three classical machine learning algorithms—Logistic Regression, k-Nearest Neighbors (k-NN) and Random Forest—were compared using raw and PCA-compressed features. Further experiments examined numerical stability, model capacity, bootstrap variability, gradient-based optimization and performance at different training-set sizes.

The results show that PCA retained 78.20% of the variance using 50 components. Random Forest achieved 81.33% accuracy on raw features in the baseline comparison, while k-NN achieved 76.33% accuracy using 50 PCA components. The capacity experiment demonstrated that increasing tree depth improved training accuracy but eventually increased the generalization gap. A numerically stable softmax implementation avoided overflow, and the deployment diagnostic measured a serialized Random Forest size of approximately 2.041 MB.

Finally, the assignment examines the implications of extending the system to 38 classes, handling approximately 20% noisy labels and meeting mobile deployment constraints of under 50 ms per image and under 5 MB. The findings support further evaluation of compact models while highlighting the need for additional experiments before claiming readiness for real-world deployment.

1. INTRODUCTION

1.1 Background

Plant diseases can reduce crop yield and quality. A computer-based crop-disease screening system can help identify possible diseases from photographs of plant leaves. AgriVision is considered a proposed system that receives a leaf image and predicts its disease category.

Before adopting a deep neural network, it is necessary to understand how data representation, feature compression, uncertainty, optimization and model capacity affect classification performance. Classical machine learning models provide useful baselines for this analysis.

1.2 Objectives

The objectives of this assignment are:

1. To analyse the problem structure and numerical representation of leaf images.
2. To examine feature redundancy using PCA and reconstruction error.
3. To quantify uncertainty using entropy, cross-entropy and KL divergence.
4. To demonstrate numerical instability in softmax and investigate feature conditioning.
5. To compare three different machine learning algorithms on raw and compressed features.
6. To examine underfitting, overfitting, bias and variance.
7. To implement softmax regression from scratch and compare optimization methods.
8. To organize model performance across different training-set sizes.
9. To relate theoretical expectations to experimental observations.
10. To assess whether deep learning is justified and propose adaptations for the changed deployment requirements.

2. DATASET AND METHODOLOGY

2.1 Dataset source

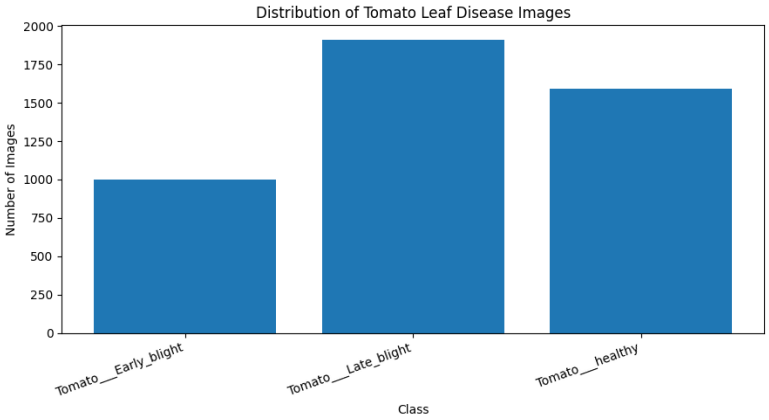
The experiment used the PlantVillage dataset available through the Hugging Face dataset repository geraldmc/plantvillage-full.

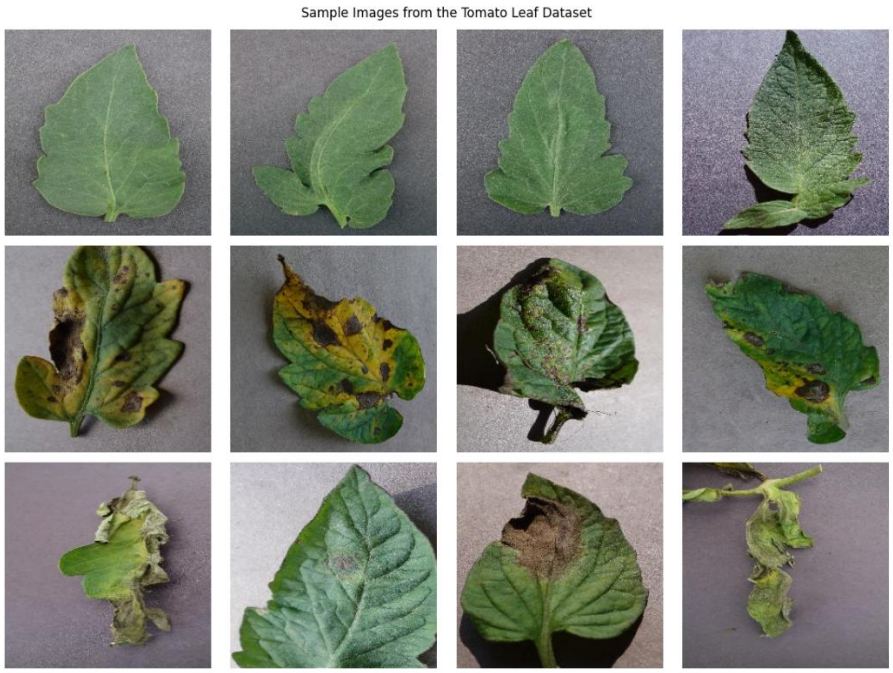
The loaded dataset contained **54,304 images and 38 crop-disease classes**. For the main experiments, three tomato classes were selected:

- Tomato Healthy
- Tomato Early Blight
- Tomato Late Blight

The filtered dataset contained 4,500 images.

Class	Number of images	Proportion
Tomato Early Blight	1,000	22.22%
Tomato Late Blight	1,909	42.42%
Tomato Healthy	1,591	35.36%
Total	**4,500**	**100%**





2.2 Image preprocessing

Each image was converted to grayscale and resized to 64 × 64 pixels. Pixel values were converted to floating-point numbers and normalized to the interval [0, 1] by dividing each intensity by 255.

The grayscale image was then flattened into a vector containing 4,096 features.

The preprocessing sequence was:

1. Select the three tomato classes.
2. Convert each image to grayscale.
3. Resize each image to 64 × 64 pixels.
4. Normalize pixel intensities by dividing by 255.
5. Flatten the image into 4,096 features.
6. Split the dataset into training, validation and test sets.

2.3 Dataset split

A stratified split was used to preserve the approximate class proportions.

Subset	Number of images	Percentage
Training	3,150	70%
Validation	675	15%
Test	675	15%
Total	**4,500**	**100%**

The training data was used for fitting the models and PCA. Validation data was used for examining generalization and optimization behaviour. The test data was intended for final evaluation.

3. TASK-WISE ANALYSIS AND RESULTS

TASK 1 – ANALYSE THE PROBLEM STRUCTURE AND DATA REPRESENTATION

3.1 Learning-task components

AgriVision is a supervised, multiclass image-classification problem. Each input image is associated with a known class label during training, and the trained model predicts a class for a previously unseen image.

Component	Description

Learning task	Supervised multiclass classification
Input	Photograph of a plant leaf
Numerical input representation	64 × 64 grayscale image, flattened into 4,096 features
Output	Healthy, Early Blight or Late Blight
Training information	Images and their known class labels
Performance measures	Accuracy, precision, recall, macro F1-score and confusion matrix
Main constraint	The model must learn useful disease patterns while generalizing to unseen images

Accuracy alone is insufficient because it does not show whether the model performs equally well on all classes. Macro F1-score gives equal weight to the F1-score of each class and is therefore useful when class proportions differ.

3.2 Scalars, vectors, matrices and tensors

The shape of a dataset depends on how the information is represented.

Data item	**Representation**	**Shape**
One normalized pixel	Scalar	One numerical value
One grayscale image	Matrix	64 × 64
One flattened image	Vector	4,096
Complete selected dataset	Matrix	4,500 × 4,096
Mini-batch of 32 grayscale images with channel dimension	Tensor	32 × 64 × 64 × 1
Class-label vector	Vector	4,500
One-hot encoded labels	Matrix	4,500 × 3

The matrix representation of the dataset makes it possible to apply conventional machine learning algorithms. The tensor representation preserves the image's spatial dimensions and is commonly used by convolutional neural networks.

Flattening allows algorithms such as Logistic Regression and k-NN to operate on images as feature vectors. However, it removes the explicit two-dimensional neighbourhood structure of the pixels. Consequently, a model operating on flattened features must learn useful relationships without directly using the image's spatial arrangement.

3.3 Comparison of vector norms

Given the vector,

$$\mathbf{x} = \begin{bmatrix} 3 \\ 4 \\ 0 \end{bmatrix}$$

1. L^1 Norm (Manhattan Norm)

The L^1 norm is the sum of the absolute values of all vector components.

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|$$

Substituting the given values,

$$\begin{aligned} \|\mathbf{x}\|_1 &= |3| + |4| + |0| \\ &= 3 + 4 + 0 \\ &= \boxed{7} \end{aligned}$$

2. L^2 Norm (Euclidean Norm)

The L^2 norm is the square root of the sum of the squares of all vector components.

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

Substituting the given values,

$$\begin{aligned}\|\mathbf{x}\|_2 &= \sqrt{3^2 + 4^2 + 0^2} \\ &= \sqrt{9 + 16 + 0} \\ &= \sqrt{25} \\ &= \boxed{5}\end{aligned}$$

3. L∞ Norm (Maximum Norm)

The L∞ norm is the maximum absolute value among all vector components.

$$\|\mathbf{x}\|_\infty = \max_{1 \le i \le n} |x_i|$$

Substituting the given values,

$$\begin{aligned}\|\mathbf{x}\|_\infty &= \max(|3|, |4|, |0|) \\ &= \max(3, 4, 0) \\ &= \boxed{4}\end{aligned}$$

Final results:

$$\boxed{\|\mathbf{x}\|_1 = 7, \quad \|\mathbf{x}\|_2 = 5, \quad \|\mathbf{x}\|_\infty = 4}$$

The L^1 norm measures the total absolute magnitude of the features. The L^2 norm measures their Euclidean magnitude and is widely used in distance calculations and regularization. The L∞ norm measures the largest absolute component and can help identify the maximum feature deviation.

For image analysis, L^2 distance is commonly associated with Euclidean similarity, while L^1 distance can be less dominated by a few large differences. The appropriate norm depends on the intended comparison and the distribution of the features.

3.4 Effect of preprocessing

Resizing: Resizing every image to 64 × 64 produces a consistent input size. However, small disease spots and fine texture details may be lost.

Grayscale conversion: Converting RGB images to grayscale reduces three colour channels to one. This reduces dimensionality but also removes colour information that may help distinguish some diseases.

Normalization: Dividing pixel values by 255 converts the intensity range from 0–255 to 0–1. This provides a consistent numerical scale and can help optimization.

Flattening: Flattening produces a 4,096-element vector suitable for conventional classifiers, but it does not explicitly preserve spatial relationships.

Train-validation-test split: Separating the data provides a way to examine generalization. Using the test set repeatedly to choose model settings can lead to optimistic estimates of performance.

Task 1 result

The selected dataset contained 4,500 images represented by a matrix of shape (4500, 4096). The preprocessing process created consistent numerical inputs and enabled the subsequent PCA and classification experiments.

```
[15]
✓ 0s
import numpy as np
from PIL import Image
from sklearn.model_selection import train_test_split

X_list = []
y_list = []

for item in tomato_data:
    img = item["image"].convert("L")
    img = img.resize((64, 64))

    # Convert pixels to values between 0 and 1
    arr = np.array(img, dtype=np.float32) / 255.0

    # Flatten 64 x 64 into 4096 features
    X_list.append(arr.flatten())
    y_list.append(item["class_label"])

X = np.array(X_list, dtype=np.float32)
y = np.array(y_list)

print("Feature matrix shape:", X.shape)
print("Label vector shape:", y.shape)
print("Minimum pixel value:", X.min())
print("Maximum pixel value:", X.max())

Feature matrix shape: (4500, 4096)
Label vector shape: (4500,)
Minimum pixel value: 0.0
Maximum pixel value: 1.0
```

TASK 2 – EXAMINE THE FEATURE SPACE USING LINEAR ALGEBRA

4.1 Principal Component Analysis

Each flattened image contains 4,096 pixel features. Many pixels may be correlated because neighbouring pixels often represent the same leaf region, background or lesion structure. PCA identifies orthogonal directions that explain the greatest variation in the training data.

PCA was fitted to the training set using a randomized solver and 100 components. The measured fitting time was approximately 5.60 seconds.

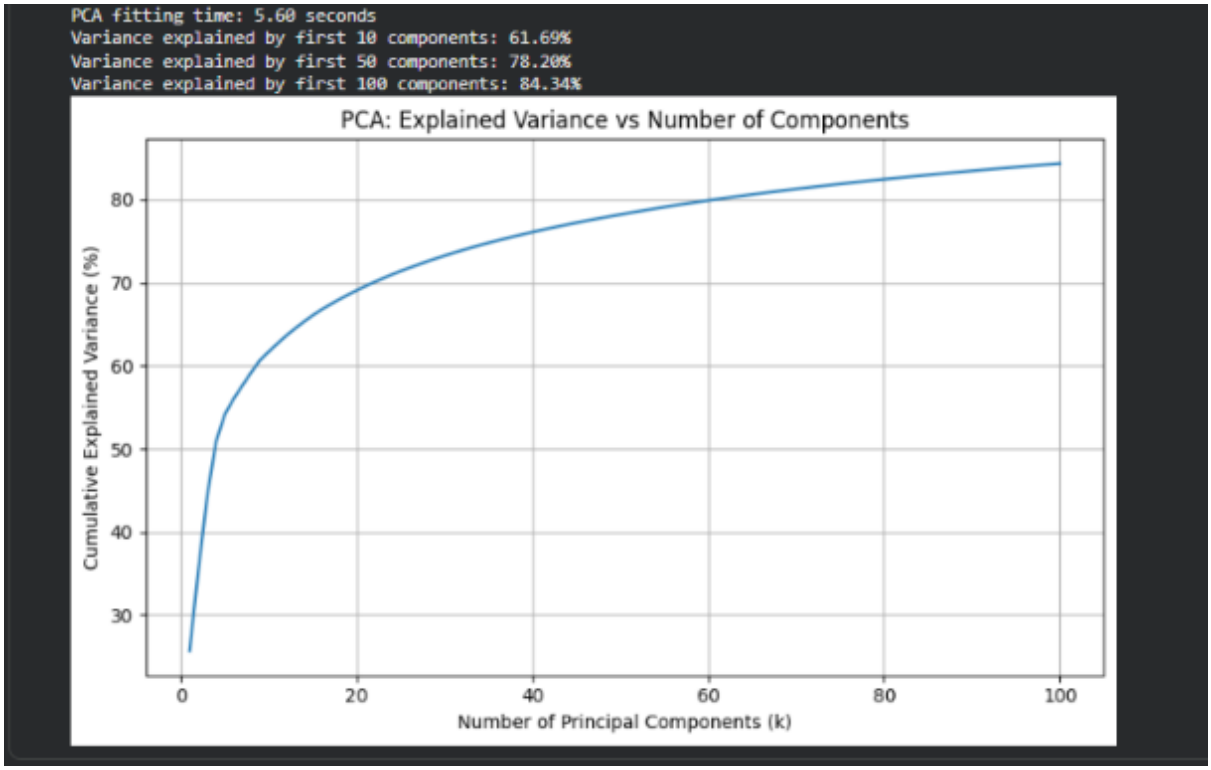
4.2 Explained variance

The cumulative explained variance obtained from PCA was:

Number of components	Cumulative explained variance
10	61.69%
50	78.20%
100	84.34%

The first 50 components retained 78.20% of the measured variance while reducing the feature count from 4,096 to 50. This is a substantial reduction in dimensionality.

However, explained variance does not directly measure classification accuracy. A component can explain substantial pixel variation without necessarily providing the most useful information for distinguishing disease categories.



4.3 Reconstruction error

Reconstruction error measures the difference between the original image vector and its approximation using a limited number of principal components.

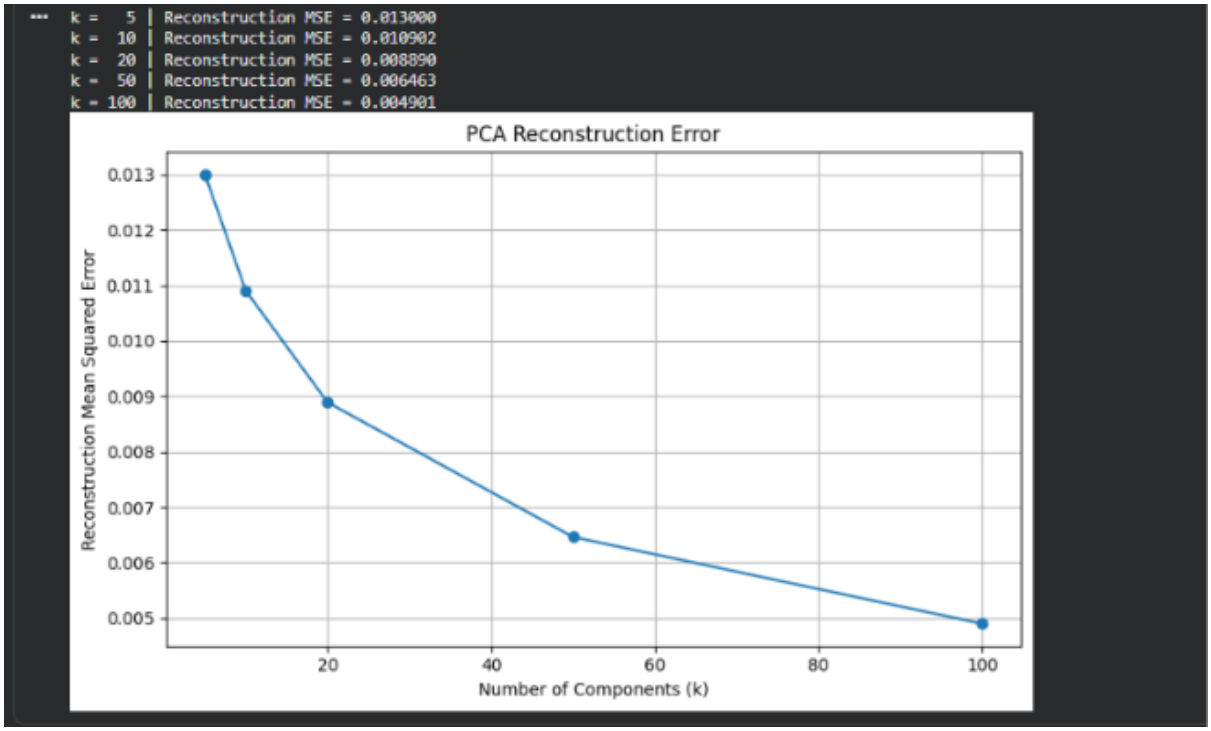
The experiment calculated mean squared error (MSE) on the test images.

Components (k)	Reconstruction MSE
5	0.013000
10	0.010902
20	0.008890
50	0.006463
100	0.004901

The error decreased as more components were retained. Using only five components produced an MSE of 0.013000, whereas 100 components reduced it to 0.004901.

The improvement becomes smaller as additional components are included. Fifty components provide a useful compression point for comparison because they reduce

4,096 features to 50 while retaining 78.20% of the variance. This is a practical trade-off for the experiments, although it is not necessarily the optimal compression level for every classifier.

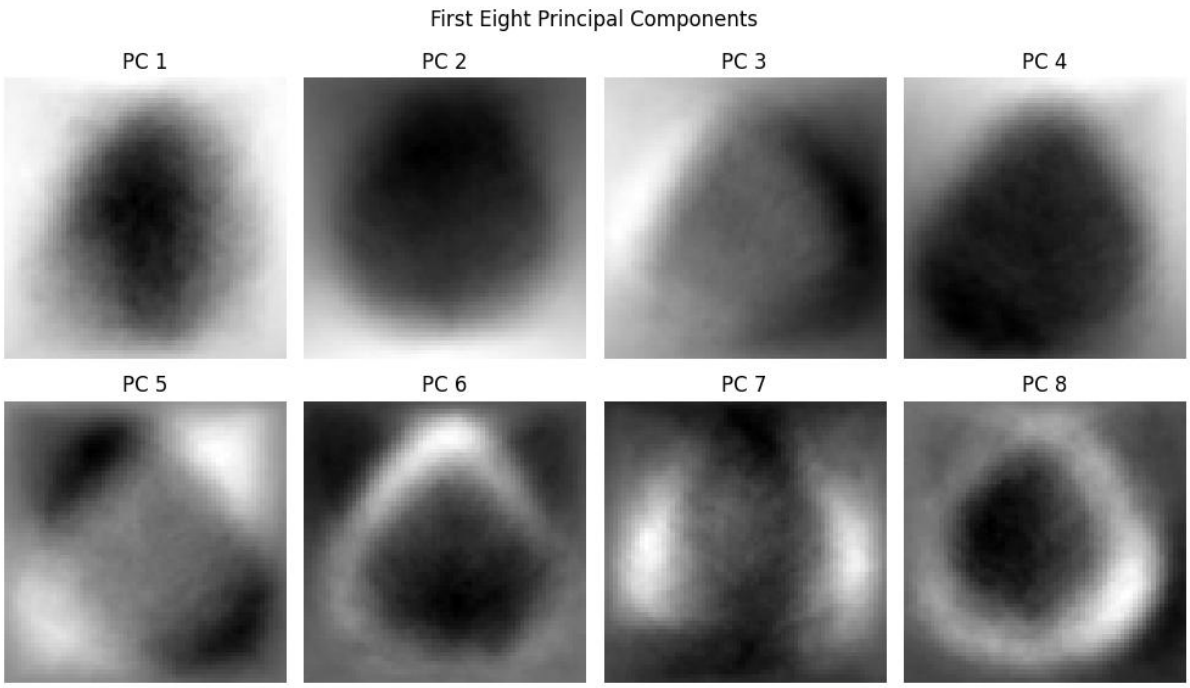


4.4 Principal-component images

The first eight principal components were reshaped into 64 × 64 arrays and visualized as grayscale images.

These component images represent directions of variation in the dataset. They are not individual disease photographs. Bright and dark regions indicate positive and negative contributions to a component, depending on the component's sign.

The leading components appear to capture broad image structure and variations in leaf-region intensity. Later components capture more localized patterns of variation.



4.5 Relationship between eigenvalues and singular values

For a centered data matrix (X_c), singular value decomposition gives:

$$X_c = U\Sigma V^T$$

The singular values are the diagonal entries of (\Sigma). The eigenvalues of the sample covariance matrix are proportional to the squared singular values:

$$\lambda_i = \frac{\sigma_i^2}{n-1}$$

where (n) is the number of training observations.

Explained variance is proportional to the eigenvalue associated with each component. A rapid decrease in the eigenvalue spectrum indicates that a relatively small number of directions captures a large proportion of the total variance.

The cumulative explained-variance results are consistent with substantial redundancy among the original pixel features. However, the notebook did not print the complete singular-value spectrum or the full covariance eigenvalues.

4.6 Hand-worked eigendecomposition example

Consider the illustrative symmetric matrix:

Given the covariance-like matrix:

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Step 1: Find the eigenvalues

The eigenvalues are obtained by solving the characteristic equation:

$$\det(A - \lambda I) = 0$$

Substituting A and the identity matrix I:

$$\begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix} = 0$$

Expanding the determinant:

$$(2-\lambda)(2-\lambda) - (1)(1) = 0$$

$$(2-\lambda)^2 - 1 = 0$$

Expanding and simplifying:

$$4 - 4\lambda + \lambda^2 - 1 = 0$$

$$\lambda^2 - 4\lambda + 3 = 0$$

Factoring the quadratic equation:

$$(\lambda - 3)(\lambda - 1) = 0$$

Therefore, the eigenvalues are:

$$\boxed{\lambda_1 = 3, \qquad \lambda_2 = 1}$$

Step 2: Find the eigenvectors

The eigenvectors are obtained using:

$$(A - \lambda I)v = 0$$

For $\lambda_1 = 3$:

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This gives $x = y$. Choosing $x = y = 1$, the eigenvector is:

$$v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Its magnitude is:

$$\|v_1\|_2 = \sqrt{1^2 + 1^2} = \sqrt{2}$$

Therefore, the normalized eigenvector is:

$$\boxed{v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}}$$

For $\lambda_2 = 1$:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This gives $x = -y$. Choosing $x = 1$ and $y = -1$, the eigenvector is:

$$v_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Its magnitude is:

$$\|v_2\|_2 = \sqrt{1^2 + (-1)^2} = \sqrt{2}$$

Therefore, the normalized eigenvector is:

$$\boxed{v_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}}$$

Step 3: Calculate the explained variance

The total sum of eigenvalues is:

$$\lambda_{\text{total}} = \lambda_1 + \lambda_2$$

$$\lambda_{\text{total}} = 3 + 1 = 4$$

The percentage of variance explained by the first principal component is:

$$\text{Explained Variance}_1 = \frac{\lambda_1}{\lambda_1 + \lambda_2} \times 100$$

$$= \frac{3}{3+1} \times 100$$

$$\boxed{\text{Explained Variance}_1 = 75\%}$$

Similarly, the percentage of variance explained by the second principal component is:

$$\text{Explained Variance}_2 = \frac{1}{3+1} \times 100$$

$$\boxed{\text{Explained Variance}_2 = 25\%}$$

of the total variance, while the second accounts for 25%.

This small example illustrates how eigendecomposition identifies directions with different amounts of variation. It is an illustrative mathematical example rather than a decomposition of the tomato dataset.

4.7 Computational cost

PCA becomes more expensive as the number of observations and features increases. The original data matrix contains 4,096 features per image, while the compressed representation contains only 50 features in the model comparison.

Although fitting PCA has an initial computational cost, subsequent models can operate on the lower-dimensional representation. This can reduce training and prediction costs for some algorithms.

Task 2 result

PCA reduced the representation from 4,096 to 50 features while retaining 78.20% of the measured variance. Reconstruction error decreased consistently as more components were included. Fifty components were selected as a practical compression level for the later classifier comparisons.

TASK 3 – ANALYSE UNCERTAINTY USING PROBABILITY AND INFORMATION THEORY

5.1 Class probabilities

The three class probabilities were calculated from the 4,500 selected images.

Class	Probability
Early Blight	0.2222
Late Blight	0.4242
Healthy	0.3536

The probabilities are not identical, because Late Blight is the most frequent class and Early Blight is the least frequent.

5.2 Class entropy

Shannon entropy measures uncertainty in a discrete probability distribution:

$$H(Y) = -\sum_{i=1}^{k} p_i \log_2(p_i)$$

Using the observed class proportions, the notebook produced:

$$\boxed{H(Y) = 1.5373 \text{ bits}}$$

The maximum possible entropy for three equally likely classes is:

$$H_{\max} = \log_2(3)$$

$$\boxed{H_{\max} = 1.5850 \text{ bits}}$$

The measured entropy is relatively close to the maximum, indicating that the class distribution is fairly balanced overall, although some imbalance remains.

5.3 Entropy, cross-entropy and KL divergence

To demonstrate the relationship between these quantities, the notebook used the following illustrative probability distributions:

The probability distributions are:

$$P = [0.7, 0.2, 0.1]$$

$$Q = [0.5, 0.3, 0.2]$$

The entropy of P is:

$$H(P) = -\sum_i p_i \log_2(p_i)$$

$$H(P) = -[0.7 \log_2(0.7) + 0.2 \log_2(0.2) + 0.1 \log_2(0.1)]$$

$$\boxed{H(P) = 1.1568 \text{ bits}}$$

Cross-entropy measures the cost of representing the true distribution (P) using the predicted distribution (Q):

The cross-entropy of P and Q is:

$$H(P, Q) = -\sum_i p_i \log_2(q_i)$$

$$H(P, Q) = -[0.7 \log_2(0.5) + 0.2 \log_2(0.3) + 0.1 \log_2(0.2)]$$

$$\boxed{H(P, Q) = 1.2796 \text{ bits}}$$

KL divergence measures the difference between the distributions:

$$D_{\mathrm{KL}}(P \| Q) = \sum_i p_i \log_2 \left(\frac{p_i}{q_i} \right)$$

$$D_{\mathrm{KL}}(P \| Q) = 0.7 \log_2 \left(\frac{0.7}{0.5} \right) + 0.2 \log_2 \left(\frac{0.2}{0.3} \right) + 0.1 \log_2 \left(\frac{0.1}{0.2} \right)$$

$$\boxed{D_{\mathrm{KL}}(P \| Q) = 0.1228 \text{ bits}}$$

KL divergence measures the difference between the distributions:

$$D_{\mathrm{KL}}(P \| Q) = \sum_i p_i \log_2 \left(\frac{p_i}{q_i} \right)$$

$$D_{\mathrm{KL}}(P \| Q) = 0.7 \log_2 \left(\frac{0.7}{0.5} \right) + 0.2 \log_2 \left(\frac{0.2}{0.3} \right) + 0.1 \log_2 \left(\frac{0.1}{0.2} \right)$$

$$\boxed{D_{\mathrm{KL}}(P \| Q) = 0.1228 \text{ bits}}$$

In the opposite direction:

$$D_{\mathrm{KL}}(Q \| P) = \sum_i q_i \log_2 \left(\frac{q_i}{p_i} \right)$$

$$D_{\mathrm{KL}}(Q \| P) = 0.5 \log_2 \left(\frac{0.5}{0.7} \right) + 0.3 \log_2 \left(\frac{0.3}{0.2} \right) + 0.2 \log_2 \left(\frac{0.2}{0.1} \right)$$

$$\boxed{D_{\mathrm{KL}}(Q \| P) = 0.1328 \text{ bits}}$$

The values differ because KL divergence is asymmetric. (D_{\mathrm{KL}}(P|Q)) measures the discrepancy when (P) is treated as the reference distribution and (Q) as

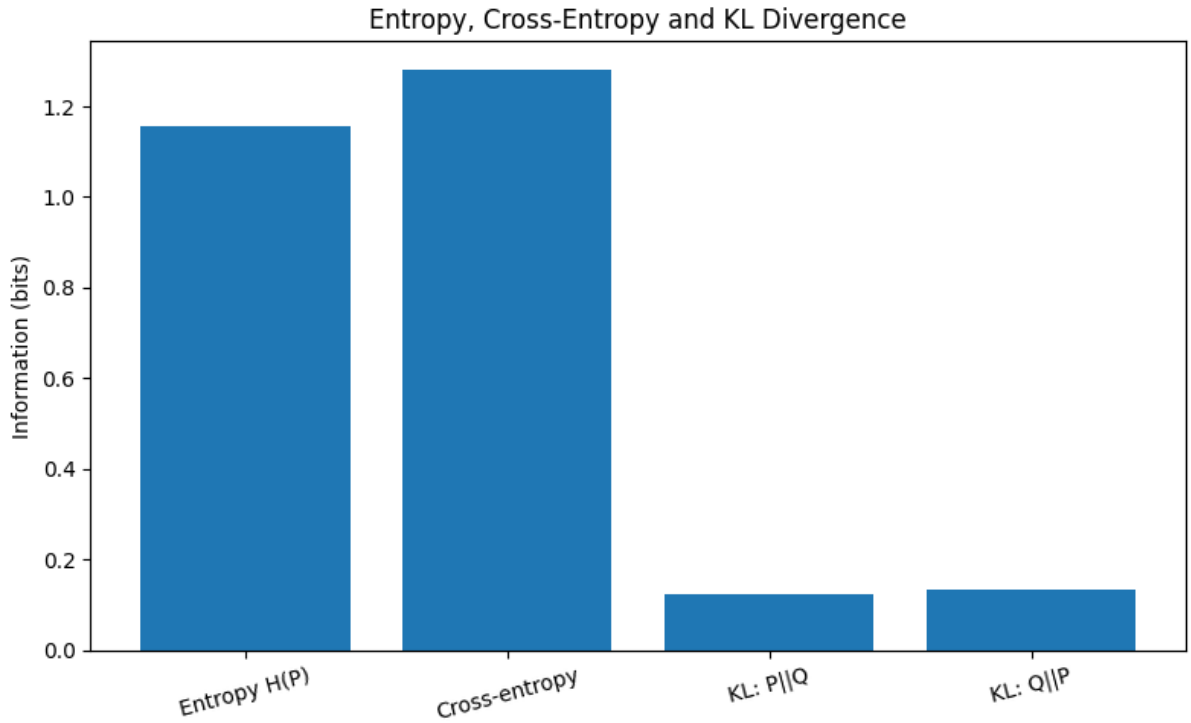
the approximation. Reversing the distributions changes the weights assigned to each category.

The cross-entropy decomposition was verified numerically:

$$H(P,Q) = H(P) + D_{\mathrm{KL}}(P\|Q)$$

$$1.2796 \approx 1.1568 + 0.1228$$

If KL divergence is zero, the two distributions are identical for all categories with positive probability in the reference distribution.



5.4 Connection to classifier behaviour

For a fixed true distribution, (H(P)) is constant. Consequently, minimizing cross-entropy also minimizes (D_{\mathrm{KL}}(P|Q)). Both objectives encourage the predicted probabilities to approach the true probabilities.

The class entropy measured from the tomato dataset is an actual dataset result. The cross-entropy and KL-divergence numbers above are an illustrative numerical demonstration, not measured losses from the trained Logistic Regression, k-NN or Random Forest models. A direct comparison of those models' predicted probability distributions was not completed in the notebook.

Task 3 result

The observed class entropy was 1.5373 bits out of a maximum of 1.5850 bits. The numerical example verified the cross-entropy decomposition and showed that KL divergence is asymmetric.

TASK 4 – DIAGNOSE NUMERICAL COMPUTATION ISSUES

6.1 Failure of naive softmax

Softmax converts a vector of logits into probabilities:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

The notebook tested:

$$z = [1000, 999, 995]$$

Exponentiating these large values exceeds the floating-point range. The naive softmax calculation produced:

Naive softmax: [nan nan nan]

This demonstrates numerical overflow. When the exponentials become infinite, dividing one infinite value by the sum of infinite values can produce undefined results.

6.2 Numerically stable softmax

A stable implementation subtracts the largest logit from each logit before exponentiation:

$$p_i = \frac{e^{z_i - z_{\max}}}{\sum_j e^{z_j - z_{\max}}}$$

For the same input, the notebook obtained:

Class	Stable softmax probability
Class 1	0.7275
Class 2	0.2676
Class 3	0.0049

The probabilities summed to approximately 1.0000.

Subtracting the same constant from every logit does not change the mathematical softmax probabilities, because the common exponential factor cancels between the numerator and denominator. It prevents overflow and improves numerical stability.

[Insert Figure 8 here: Stable softmax probability graph.]

6.3 Feature covariance condition number

The covariance matrix condition number was estimated using the first 1,000 training observations and 256 pixel features.

The measured values were:

Quantity	Result
Largest covariance eigenvalue	(4.901950)
Smallest covariance eigenvalue	(3.910390\times10^{-5})
Estimated condition number	(1.2536\times10^5)

The condition number is the ratio of the largest eigenvalue to the smallest positive eigenvalue. A large value indicates that the covariance matrix contains directions with very different scales of variation.

In gradient-based optimization, poor conditioning can cause progress to be much faster in some directions than in others. This can lead to slow convergence or sensitivity to the learning rate.

The measured condition number is a diagnostic estimate from a subset of the features, not the condition number of the full 4,096-feature covariance matrix.

Task 4 result

The naive softmax failed for large logits, while the stable version returned valid probabilities. The estimated covariance condition number was approximately (1.2536\times10^5), indicating substantial differences in feature-variation scales in the tested subset.

TASK 5 – DIFFERENTIATE BASELINE LEARNING STRATEGIES

7.1 Models compared

Three different machine learning approaches were implemented:

Logistic Regression: A linear parametric classifier that learns class decision boundaries through a weighted combination of the input features.

k-Nearest Neighbors: A non-parametric algorithm that classifies an observation based on the labels of nearby training examples. It depends on the distance representation and can be affected by high-dimensional data.

Random Forest: An ensemble of decision trees that can model nonlinear relationships and interactions between features.

The comparison used 1,500 training images and a fixed test subset of 300 images. The raw representation contained 4,096 features, while the compressed representation used the first 50 PCA components.

7.2 Measured performance

Feature representation	Model	Accuracy	Macro F1
Raw 4,096 features	Logistic Regression	57.33%	0.5670
Raw 4,096 features	k-NN	60.33%	0.6056
Raw 4,096 features	Random Forest	**81.33%**	**0.7902**
PCA, 50 components	Logistic Regression	62.33%	0.6250
PCA, 50 components	k-NN	76.33%	0.7481
PCA, 50 components	Random Forest	79.67%	0.7726

[Insert Figure 9 here: Raw-feature versus PCA-feature accuracy comparison.]

7.3 Interpretation

Random Forest achieved the highest accuracy in this baseline comparison, at 81.33% using raw features. It can learn nonlinear relationships that a linear model cannot represent directly.

PCA improved Logistic Regression accuracy from 57.33% to 62.33%. It improved k-NN accuracy from 60.33% to 76.33%, suggesting that reducing redundant or unhelpful dimensions made the distance-based classifier more effective in this experiment.

Random Forest accuracy decreased slightly from 81.33% to 79.67% after compression. This indicates that PCA did not preserve all information useful to that classifier.

The Logistic Regression experiment on raw features produced a convergence warning because the iteration limit was reached. Its score should therefore be interpreted as the result of the configured run rather than a fully optimized linear-model baseline.

7.4 Training and prediction times

Feature representation	Model	Training time (s)	Prediction time (s)
Raw	Logistic Regression	31.3465	0.0096
Raw	k-NN	0.0156	0.4167
Raw	Random Forest	7.0861	0.0364
PCA, 50 components	Logistic Regression	0.0400	0.0006
PCA, 50 components	k-NN	0.0024	0.0157
PCA, 50 components	Random Forest	0.8804	0.0346

These timings are environment-dependent and were measured on the experiment's test batch. They should not be interpreted as universal timings or as smartphone latency measurements.

PCA substantially reduced training time for Logistic Regression and Random Forest in the reported runs. It also reduced k-NN prediction time, which is important because k-NN stores and compares training observations at inference time.

Task 5 result

Random Forest on raw features achieved 81.33% accuracy and a macro F1-score of 0.7902. PCA improved Logistic Regression and k-NN performance in this experiment, but slightly reduced Random Forest accuracy. This demonstrates that feature compression affects algorithms differently.

TASK 6 – ANALYSE CAPACITY, OVERFITTING AND UNDERFITTING

8.1 Experimental method

A Decision Tree classifier was trained using 1,200 training images. Its maximum depth was varied across 1, 2, 3, 5, 10, 20 and unrestricted depth. Performance was evaluated on a separate validation subset of 500 images.

8.2 Results

Maximum depth	Training accuracy	Validation accuracy	Generalization gap
1	48.42%	46.60%	0.0182
2	55.92%	50.20%	0.0572
3	67.17%	54.80%	0.1237
5	78.42%	59.40%	0.1902

10	98.67%	**66.40%**	0.3227
20	100.00%	64.60%	0.3540
Unrestricted	100.00%	64.60%	0.3540

The generalization gap is the training accuracy minus the validation accuracy.

[Insert Figure 10 here: Training and validation error against maximum Decision Tree depth.]

8.3 Underfitting and overfitting

At shallow depths, the Decision Tree has limited capacity and cannot represent sufficiently complex decision boundaries. At depth 1, training accuracy was only 48.42%, and validation accuracy was 46.60%. This indicates underfitting.

As the maximum depth increased, training accuracy rose. At depth 10, training accuracy reached 98.67%, while validation accuracy reached its highest observed value of 66.40%.

At depth 20 and unrestricted depth, training accuracy became 100%, but validation accuracy decreased to 64.60%. The generalization gap increased to 0.3540, indicating overfitting.

This result shows that maximizing training accuracy does not necessarily maximize performance on unseen images. A model can memorize training-specific patterns that do not generalize.

Hyperparameters should be selected using validation data or cross-validation. Repeatedly selecting models using the test set introduces information from the test data into the model-selection process and can lead to an optimistic estimate of generalization.

Task 6 result

The best validation accuracy in the tested settings was 66.40% at depth 10. Deeper trees achieved perfect training accuracy but lower validation accuracy, demonstrating the effect of excessive model capacity.

TASK 7 – DECOMPOSE ERROR INTO BIAS AND VARIANCE

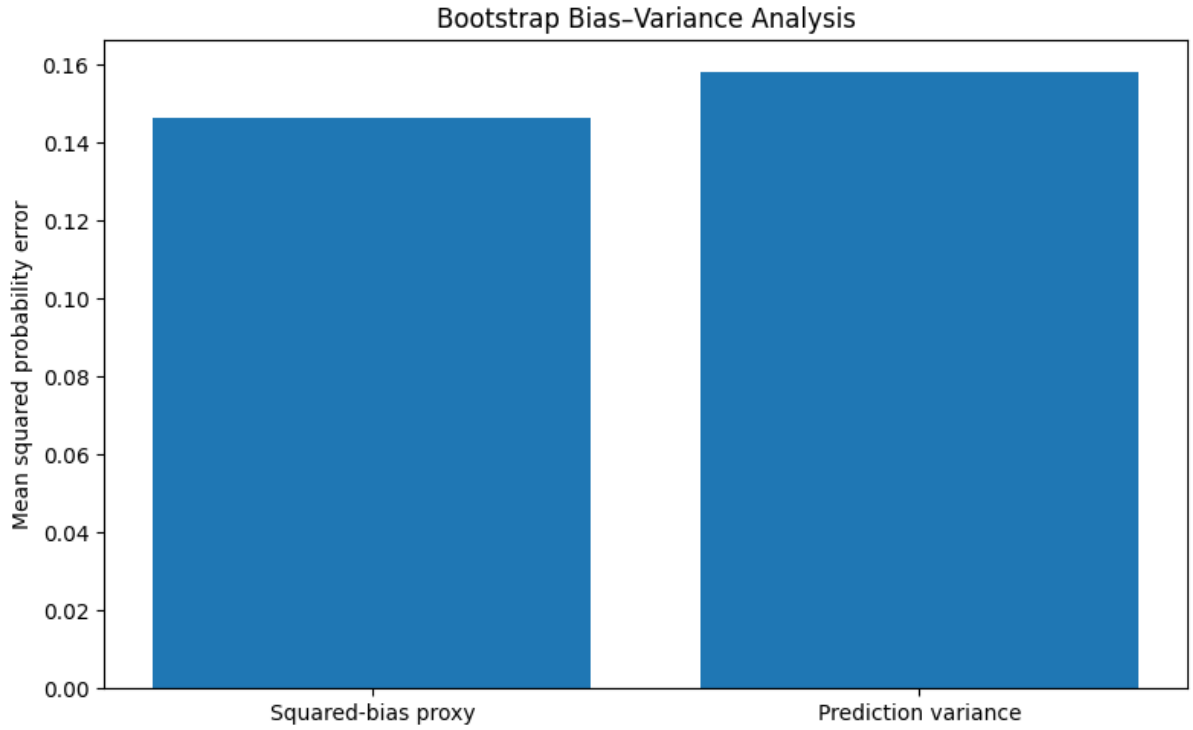
9.1 Bootstrap experiment

Bootstrap resampling was used to examine how sensitive a Decision Tree model is to changes in its training sample. Fifteen bootstrap replicas were generated, satisfying the minimum requirement of ten replicas.

Each replica sampled training observations with replacement and trained a Decision Tree with maximum depth 10. Predictions were compared on a fixed validation subset.

9.2 Results

Measure	Result
Number of bootstrap models	15
Mean validation accuracy	0.5381
Standard deviation of validation accuracy	0.0315
Squared-bias proxy	0.146204
Prediction-variance estimate	0.158340
Combined error proxy	0.304544



The estimated prediction variance was slightly larger than the squared-bias proxy. This suggests that the predictions varied noticeably across bootstrap training samples. However, these values are approximate indicators based on squared probability errors, not a complete exact decomposition of classification error.

In the standard bias–variance framework:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \sigma^2$$

where (\sigma^2) represents irreducible noise. The notebook did not independently estimate the noise term or calculate a complete low-capacity versus high-capacity decomposition. Therefore, the reported values should be interpreted as an exploratory bias–variance analysis.

9.3 Maximum-likelihood and maximum-a-posteriori estimation

Bayesian estimation provides another way to understand how prior knowledge influences an estimate.

For illustration, consider the proportion of healthy images, treating Healthy versus Not Healthy as a binary outcome. In the full selected dataset, 1,591 of 4,500 images were Healthy.

The maximum-likelihood estimate (MLE) is:

$$\hat{p}_{MLE} = \frac{k}{n}$$

$$\hat{p}_{MLE} = \frac{1591}{4500}$$

$$\boxed{\hat{p}_{MLE} \approx 0.3536}$$

Suppose a Beta(2,2) prior is selected. The maximum-a-posteriori estimate (MAP) for a binary proportion is:

$$\hat{p}_{MAP} = \frac{k + \alpha - 1}{n + \alpha + \beta - 2}$$

$$\hat{p}_{MAP} = \frac{1591 + 2 - 1}{4500 + 2 + 2 - 2}$$

$$\hat{p}_{MAP} = \frac{1592}{4502}$$

$$\boxed{\hat{p}_{MAP} \approx 0.3536}$$

The estimates are almost identical because the dataset is large relative to the prior's influence.

For a smaller illustrative sample with 7 healthy observations out of 20:

$$\hat{p}_{MLE} = \frac{k}{n} = \frac{7}{20} = 0.3500$$

$$\hat{p}_{MAP} = \frac{k + \alpha - 1}{n + \alpha + \beta - 2}$$

$$\hat{p}_{MAP} = \frac{7 + 2 - 1}{20 + 2 + 2 - 2} = \frac{8}{22}$$

$$\boxed{\hat{p}_{MAP} \approx 0.3636}$$

The prior has a more noticeable influence on the smaller sample. As the sample size grows, the data generally contributes more information and the effect of a fixed prior diminishes.

This Bayesian calculation is an additional mathematical illustration; the notebook did not execute it as part of the bootstrap experiment.

Task 7 result

The 15-replica experiment produced a mean validation accuracy of 0.5381 and a prediction-variance estimate of 0.158340. The results demonstrate sensitivity to bootstrap sampling, while the Bayesian example illustrates why prior knowledge has a greater influence on small samples.

TASK 8 – ANALYSE GRADIENT-BASED OPTIMIZATION

10.1 Softmax regression

Softmax regression was implemented from scratch using NumPy. The input features were reduced to 50 PCA components and standardized using the training subset. The model used three output classes.

For a feature vector x_i, the logits are:

$$z_i = W^T x_i + b$$

The softmax probability for class j is:

$$\hat{y}_{ij} = \frac{e^{z_{ij}}}{\sum_{k=1}^{C} e^{z_{ik}}}$$

The cross-entropy loss for one-hot target labels is:

$$\boxed{L = -\frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{C} y_{ij} \log(\hat{y}_{ij})}$$

10.2 Gradient derivation

For softmax combined with cross-entropy, the derivative with respect to each logit is:

$$\frac{\partial L}{\partial z_{ij}} = \frac{1}{N}(\hat{y}_{ij} - y_{ij})$$

Applying the chain rule to the weight matrix gives:

$$\boxed{\frac{\partial L}{\partial W} = \frac{1}{N} X^T (\hat{Y} - Y)}$$

The bias gradient is:

$$\boxed{\frac{\partial L}{\partial b} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)}$$

The gradient-descent updates are:

$$\boxed{W \leftarrow W - \eta \frac{\partial L}{\partial W}}$$

$$\boxed{b \leftarrow b - \eta \frac{\partial L}{\partial b}}$$

where (\eta) is the learning rate. The error term (\hat{Y}-Y) drives the update: when predicted probabilities differ from the true labels, the gradients adjust the weights and biases.

10.3 Optimization methods

Three methods were implemented:

- **Batch Gradient Descent:** Uses the entire training subset for each update. It performs fewer updates per epoch, and its loss trajectory is generally less noisy.
- **Stochastic Gradient Descent (SGD):** Updates the parameters using one observation at a time. It performs many inexpensive updates, but its loss trajectory can be noisy.

- **Mini-batch SGD:** Uses batches of 64 observations per update. It provides a compromise between update cost and the variability of individual-sample updates.

The models were trained for 20 epochs using learning rates of 0.001, 0.01 and 0.1.

10.4 Experimental results

The configurations with the lowest final validation losses were:

Method	Learning rate	Final validation loss	Validation accuracy	Training time (s)
SGD	0.001	**0.8247**	62.67%	4.6799
Mini-batch SGD	0.1	0.8267	61.67%	0.1526
SGD	0.01	0.8794	59.33%	3.3665
Mini-batch SGD	0.01	0.8799	62.00%	0.1383
Batch GD	0.1	0.9314	62.67%	0.0415
Mini-batch SGD	0.001	1.0494	61.00%	0.1313
Batch GD	0.01	1.0712	60.67%	0.0306
Batch GD	0.001	1.0957	61.00%	0.0627
SGD	0.1	2.2845	57.00%	3.1206

[Insert Figure 12 here: Validation cross-entropy curves for the three optimization methods and three learning rates.]

10.5 Interpretation

The lowest validation loss was obtained using SGD with a learning rate of 0.001. It achieved a validation loss of 0.8247 and accuracy of 62.67%.

SGD with a learning rate of 0.1 produced a substantially higher loss of 2.2845 and lower accuracy of 57.00%. This suggests that the large learning rate caused unstable or inefficient optimization under the tested configuration.

Mini-batch SGD with a learning rate of 0.1 achieved a validation loss close to the best result while requiring approximately 0.153 seconds of training time. Its observed training time was much lower than that of SGD because it performed fewer updates per epoch.

The results show that the optimization method and learning rate affect both the loss trajectory and the computational cost. The measured covariance condition number in Task 4 provides a possible explanation for sensitivity to the learning rate, although this experiment does not establish a direct causal relationship between that particular condition-number estimate and every observed loss curve.

Task 8 result: SGD with a learning rate of 0.001 produced the lowest measured validation loss, while mini-batch SGD offered a favourable training-time trade-off.

TASK 9 – ORGANISE AND EXAMINE EXPERIMENTAL PERFORMANCE

11.1 Dataset scales

The assignment proposes experiments at 1,000, 5,000, 10,000 and full-dataset images. The filtered dataset contains 4,500 images, and the training split contains 3,150 images. Therefore, experiments at 5,000 and 10,000 training images could not be performed using the current three-class training split.

Instead, the notebook evaluated 1,000, 2,000 and 3,150 training images, using the same test set and PCA-compressed features.

11.2 Results across training-set sizes

Training images	Model	Accuracy	Macro precision	Macro recall	Macro F1
1,000	Logistic Regression	0.6193	0.6213	0.6160	0.6182
1,000	k-NN	0.7659	0.7527	0.7626	0.7519
1,000	Random Forest	0.7807	0.7724	0.7443	0.7531
2,000	Logistic Regression	0.6400	0.6390	0.6352	0.6370
2,000	k-NN	0.7704	0.7549	0.7591	0.7536
2,000	Random Forest	0.8089	0.7942	0.7805	0.7859
3,150	Logistic Regression	0.6430	0.6424	0.6372	0.6396

3,150	k-NN	0.8059	0.7913	0.7978	**0.7925**
3,150	Random Forest	0.8089	0.7942	0.7795	0.7852

[Insert Figure 13 here: Test accuracy against training-set size for all three models.]

The results indicate that increasing the training-set size generally improved performance, although the changes differed by model. Logistic Regression improved gradually. k-NN increased from 76.59% accuracy at 1,000 images to 80.59% at 3,150 images. Random Forest increased from 78.07% to 80.89% and then remained at approximately the same accuracy between 2,000 and 3,150 images.

11.3 Confusion matrix and class errors

The notebook generated a confusion matrix for the model selected by macro F1-score from the full-training-size results.

The selected model was k-NN, with:

- Test accuracy: 0.8059
- Macro F1-score: 0.7925

[Insert Figure 14 here: Confusion matrix for the selected k-NN model.]

The confusion matrix identifies which classes are predicted correctly and which are confused with one another. Confusion between visually similar disease classes may result from shared lesion patterns, image backgrounds or variation in disease appearance. Specific class-pair conclusions should be based on the actual confusion-matrix counts.

11.4 Training time and inference time

The measured training and inference times were:

Training images	Model	Training time (s)	Inference time (s)
1,000	Logistic Regression	0.2539	0.0031
1,000	k-NN	0.0043	0.0616
1,000	Random Forest	1.0712	0.0572
2,000	Logistic Regression	0.1218	0.0026
2,000	k-NN	0.0048	0.0413
2,000	Random Forest	1.7090	0.0435

3,150	Logistic Regression	0.1869	0.0007
3,150	k-NN	0.0037	0.0488
3,150	Random Forest	2.7488	0.0633

The timing results are specific to the Colab execution environment. Inference time can vary with hardware, implementation and batch size. The values should therefore be used for relative comparison within this experiment rather than as fixed deployment guarantees.

The serialized model-size measurement was performed separately for the Random Forest in Task 11. Model parameter count and peak runtime memory were not measured for every model.

Task 9 result

The best macro F1-score at the full training size was achieved by k-NN, at 0.7925. Random Forest achieved the highest accuracy in that table, at 80.89%. The experiment shows that the preferred model depends on whether accuracy, macro F1, training time or inference cost is prioritized.

The requested 5,000- and 10,000-image experiments remain untested because the current training split contains only 3,150 images. In addition, the best model in this table was selected using test-set performance, so the result may be optimistic. Final model selection should use validation data, with the test set reserved for the final evaluation.

TASK 10 – COMPARE THEORY WITH EXPERIMENTAL BEHAVIOUR

12.1 Explained variance and retained accuracy

PCA retained 78.20% of the variance using 50 components. However, classification accuracy did not change in direct proportion to explained variance.

Logistic Regression improved from 57.33% to 62.33%, and k-NN improved from 60.33% to 76.33%. Random Forest decreased slightly from 81.33% to 79.67%.

This agrees with the theoretical distinction between reconstructing data variation and retaining information useful for classification. PCA preserves directions of high variance, but those directions are not guaranteed to be the most discriminative for disease classification.

12.2 Optimization speed and conditioning

The estimated covariance condition number was (1.2536\times10^5) for the selected subset of 256 features. The softmax experiment also showed that learning rate affects

convergence. SGD with learning rate 0.001 produced a lower final validation loss than SGD with learning rate 0.1.

These findings are consistent with the idea that optimization depends on the feature representation and learning rate. However, the experiments did not directly measure the Hessian condition number of the softmax loss, so the covariance condition number should be treated as a diagnostic indicator rather than a complete explanation of optimizer behaviour.

12.3 Bias–variance and model capacity

The Decision Tree experiment showed that deeper trees reduced training error but eventually increased validation error. Training accuracy reached 100% at depth 20, while validation accuracy decreased to 64.60%.

The bootstrap experiment produced a prediction-variance estimate of 0.158340 and a squared-bias proxy of 0.146204. These results indicate noticeable prediction variability. However, the bootstrap and capacity experiments used different sample sizes and do not form an exact numerical decomposition of the same error.

12.4 Causes of disagreement

Differences between theoretical expectations and observed results can arise from:

- Information discarded during grayscale conversion and PCA.
- Limited training data relative to the original feature count.
- Different inductive biases of the three algorithms.
- Optimization settings and incomplete convergence.
- Sampling variability in the training and test subsets.
- The use of approximate bias–variance indicators rather than a complete decomposition.

Task 10 result

The experiments broadly support the expected relationships between compression, model capacity, numerical stability and generalization. However, explained variance does not directly predict classification accuracy, and the condition-number and bootstrap analyses provide diagnostic evidence rather than complete causal explanations.

TASK 11 – ANALYSE WHETHER DEEP LEARNING IS JUSTIFIED

13.1 Limitations of classical machine learning

The best baseline results were encouraging but not sufficient to establish a production-ready screening system. Random Forest achieved 81.33% accuracy in the raw-feature comparison, while the best macro F1-score in the dataset-size experiment was 0.7925 for k-NN using PCA features.

These results show that classical algorithms can learn useful disease patterns. However, they also leave room for improvement, and the performance may vary across classes.

The Logistic Regression model on raw features did not converge within the configured iteration limit. The k-NN model was sensitive to feature representation, while Random Forest required more training time than the other models in the dataset-size experiment.

13.2 Curse of dimensionality

The original image representation contained 4,096 features, while the training set contained 3,150 images. In high-dimensional spaces, distance measures can become less informative, and the amount of data needed to estimate reliable decision boundaries can increase.

PCA reduced the feature count to 50. The improvement in k-NN accuracy from 60.33% to 76.33% indicates that compression was beneficial in the tested setting.

13.3 Local constancy and smoothness

k-NN relies on the assumption that nearby feature vectors tend to have similar class labels. Leaf images can differ due to lighting, orientation, background and lesion appearance. Consequently, two images of the same disease may be far apart in pixel space, while different diseases may have similar low-level pixel patterns.

The measured improvement after PCA suggests that the choice of feature space affects the usefulness of neighbourhood-based classification.

13.4 Manifold structure and hierarchical representations

Natural images do not occupy every possible point in the 4,096-dimensional pixel space. They contain structured patterns associated with leaf shape, texture and lesions. PCA captures linear directions of variation but cannot fully represent all complex nonlinear relationships.

A convolutional neural network can learn hierarchical image features. Early layers can detect edges and textures, while deeper layers can combine these patterns into more complex representations. Such learned spatial features are a reasonable candidate for improving disease classification.

However, this assignment did not train and evaluate a CNN. Therefore, the current results justify evaluating deep learning as a next step; they do not prove that a CNN will outperform the classical baselines.

13.5 Compute and interpretability trade-offs

Classical models provide useful baselines and can be relatively simple to train or inspect. PCA can reduce the feature count and computational requirements of some algorithms.

Deep learning may offer stronger feature learning but typically requires additional training, model selection and deployment testing. Its predictions may also require interpretability methods if users need explanations for a disease prediction.

Task 11 result

The experiments justify further evaluation of deep learning because the classical baselines remain imperfect and the feature representation affects performance. A compact CNN should be compared with the existing baselines using the same data split and metrics before claiming that it is the better solution.

TASK 12 – DAY-4 DESIGN CHALLENGE: ANALYSE CHANGED REQUIREMENTS

The design challenge introduces four new conditions:

1. The system must classify 38 crop-disease classes.
2. Newly collected images contain approximately 20% noisy labels.
3. Inference must take less than 50 ms per image.
4. The model must fit within 5 MB.

These requirements change the classification problem, data-quality assumptions and deployment environment. The original three-class experiment cannot establish compliance with all the new conditions.

14.1 Part A – Impact analysis

Increasing the number of classes

The original experiment used three tomato classes. The changed system must distinguish 38 classes, increasing the complexity of the label space.

For equally likely classes, maximum class entropy is:

$$H_{\max} = \log_2(C)$$

For three classes:

$$H_{\max} = \log_2(3)$$

$$\boxed{H_{\max} = 1.5850 \text{ bits}}$$

For 38 classes:

$$H_{\max} = \log_2(38)$$

$$\boxed{H_{\max} \approx 5.2479 \text{ bits}}$$

The larger label space creates more possible categories and can increase classification difficulty. However, the actual entropy depends on the distribution of class frequencies, not only the number of classes.

Noisy labels

Approximately 20% noisy labels means that some training images may have incorrect class labels. A model trained on such data may learn misleading relationships and produce less reliable decision boundaries.

Label noise can increase estimated error and affect bias and variance. The effect depends on whether the noise is random or concentrated in specific classes. It may also make validation and test design more important because performance estimates must reflect reliable ground-truth labels.

Mobile constraints

The original experiments were performed in Colab. Training and prediction times measured there do not directly establish performance on a low-cost smartphone.

The model must satisfy both a latency limit and a memory limit under the intended deployment conditions. Model-file size is not identical to peak runtime memory because runtime frameworks, temporary tensors and intermediate computations also require memory.

14.2 Part B – Examination of the original conclusion

The Task 11 deployment diagnostic measured the following for a Random Forest trained on the 1,500-image raw-feature subset:

Measure	Measured result
Test accuracy	81.33%
Training time	9.6688 seconds
Prediction time for 300 images	0.0531 seconds
Average prediction time per image in the batch	0.1768 ms
Serialized model size	2.041 MB

The serialized model file was below 5 MB, and the average batch prediction time measured in Colab was below 50 ms per image. These observations suggest that the tested model may be a promising lightweight baseline.

However, these measurements do not prove that the system meets the deployment constraints. The timing was measured on Colab rather than on a low-cost smartphone, and the 2.041 MB serialized file is not a measurement of peak runtime memory. The diagnostic also used a three-class model rather than the required 38-class system.

The behaviour of the model may change when the label space expands or noisy labels are introduced. In particular, additional classes can increase confusion between visually similar diseases, while noisy labels can reduce the reliability of the learned decision boundaries.

14.3 Part C – Analysis of adaptation options

Option 1: Change the PCA dimension

The original experiments used 50 PCA components. Reducing the number of components further may reduce computational cost and memory requirements, but it can also increase reconstruction error and discard useful disease information.

Increasing the number of components may preserve more information but increase the cost of downstream computation. The appropriate dimension should therefore be selected using validation accuracy, macro F1-score, model size and inference-time measurements.

Option 2: Use noise-aware training

Label smoothing can prevent the model from assigning excessively confident probabilities to a single class. Noise-robust loss functions and confidence-based review of suspicious labels can also reduce the impact of incorrect annotations.

These approaches may improve generalization when labels are noisy, but they must be tested against a clean validation set. Excessive smoothing or aggressive filtering may remove useful learning signals.

Option 3: Compress or simplify the model

Model compression methods include quantization, pruning and knowledge distillation. Quantization can reduce the storage requirements of model parameters, while pruning removes some model connections or components. Knowledge distillation trains a smaller model to imitate a larger teacher model.

A compact, quantized CNN is a reasonable candidate for mobile image classification because it can learn spatial features while controlling model size. However, compression may reduce accuracy, and the final model must be measured on the target device.

14.4 Part D – Evidence-based conclusion

The original conclusion should be **modified rather than retained unchanged**.

The classical models provide useful baselines. PCA reduced 4,096 features to 50, and the best raw-feature Random Forest achieved 81.33% accuracy in the baseline comparison. The deployment diagnostic measured a serialized Random Forest size of 2.041 MB and an average batch prediction time of 0.1768 ms per image in Colab.

Nevertheless, these results do not establish that the original three-class model is suitable for a 38-class, noisy-label, smartphone deployment. The required on-device latency, peak memory use and robustness to approximately 20% label noise remain untested.

The next design should retain the existing classical baselines for comparison while evaluating a compact, quantized CNN. The evaluation should use all 38 classes, include controlled noisy-label experiments, and measure performance on the intended smartphone hardware.

The final system should only be accepted if it meets the following criteria:

- Correctly evaluates all 38 classes.
- Demonstrates acceptable macro F1-score and per-class recall.
- Is evaluated under approximately 20% label noise.
- Has a model size below 5 MB.
- Achieves less than 50 ms inference latency per image on the target device.
- Maintains acceptable predictive performance after compression.

These are proposed acceptance criteria, not results already achieved by the current notebook.

4. OVERALL CONCLUSION

This assignment examined the mathematical foundations and experimental behaviour of a crop-disease classification system using 4,500 tomato leaf images from the PlantVillage dataset.

The preprocessing stage converted the images into 4,096-dimensional normalized grayscale feature vectors. PCA reduced the representation to 50 components while retaining 78.20% of the variance. Reconstruction error decreased as the number of components increased, demonstrating the trade-off between information retention and dimensionality reduction.

The comparison of classical models showed that Random Forest achieved 81.33% accuracy on raw features, while PCA improved Logistic Regression and k-NN in the tested configuration. The Decision Tree capacity experiment demonstrated that deeper models can achieve perfect training accuracy while producing lower validation accuracy, indicating overfitting.

The information-theoretic calculations quantified class uncertainty and verified the relationship between entropy, cross-entropy and KL divergence using an illustrative probability example. The numerical stability experiment demonstrated that naive softmax can fail for large logits, whereas the stable formulation produces valid probabilities. The covariance condition-number estimate also indicated substantial variation in feature scales.

The softmax regression experiment showed that the optimization method and learning rate affect convergence and training cost. The dataset-size experiment indicated that performance generally improved as more training images were used, although the best model depended on the metric being considered.

Overall, the results show that classical machine learning provides meaningful baselines, but the limitations of the current feature representation and generalization performance justify further evaluation of deep learning. For the changed deployment requirements, the most appropriate next step is to compare a compact, quantized CNN with the existing baselines under realistic label-noise and smartphone conditions. Deployment should only be approved after the required accuracy, model-size and latency measurements have been verified.

5. LIMITATIONS AND FUTURE WORK

The following limitations should be considered when interpreting the results:

1. The main classification experiments used only three tomato classes from the larger dataset.
2. The raw-versus-PCA model comparison used a 1,500-image training subset and a 300-image test subset.
3. The dataset-size experiment used 1,000, 2,000 and 3,150 training images; the requested 5,000- and 10,000-image scales could not be tested with the current split.
4. The cross-entropy and KL-divergence example used illustrative distributions rather than probabilities predicted by the trained classifiers.
5. The bootstrap experiment examined one Decision Tree capacity setting and did not complete the full low-capacity versus high-capacity decomposition.
6. The numerical conditioning estimate used only 256 pixel features and 1,000 training observations.
7. The experiments did not train a CNN or deliberately introduce 20% label noise.
8. Model size and latency were measured in Colab rather than on the intended low-cost smartphone.

Future work should complete the missing probability comparisons and bias–variance experiments, evaluate additional training-data scales, train a compact CNN, test controlled label noise and benchmark the final model on target mobile hardware.

REFERENCES

1. PlantVillage image dataset, Hugging Face Datasets repository: geraldmc/plantvillage-full. Available at: https://huggingface.co/datasets/geraldmc/plantvillage-full
2. Scikit-learn documentation. *Principal Component Analysis (PCA)*. Available at: https://scikit-learn.org/stable/modules/decomposition.html#pca
3. Scikit-learn documentation. *Classification metrics*. Available at: https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics
4. NumPy documentation. *Linear algebra*. Available at: https://numpy.org/doc/stable/reference/routines.linalg.html
5. CSA47 – Deep Learning, CO1 Assignment 1: *Analyse the Mathematical and Machine Learning Foundations of a Crop-Disease Screening System*.

APPENDIX: FIGURE PLACEMENT CHECKLIST

Insert the screenshots from your Colab notebook at the corresponding locations:

1. **Figure 1:** Distribution of the three tomato classes.
2. **Figure 2:** Grid of sample Healthy, Early Blight and Late Blight images.
3. **Figure 3:** Preprocessed feature matrix, label vector and pixel range.
4. **Figure 4:** PCA cumulative explained-variance graph.
5. **Figure 5:** PCA reconstruction-error graph.
6. **Figure 6:** First eight principal-component images.
7. **Figure 7:** Entropy, cross-entropy and KL-divergence graph.
8. **Figure 8:** Numerically stable softmax probability graph.
9. **Figure 9:** Accuracy comparison for raw and PCA features.
10. **Figure 10:** Decision Tree training and validation error curves.
11. **Figure 11:** Bootstrap bias–variance graph.
12. **Figure 12:** Softmax optimization loss curves.
13. **Figure 13:** Accuracy versus training-set size graph.
14. **Figure 14:** Confusion matrix for the selected k-NN model.

Include the relevant printed output tables for Tasks 2, 3, 4, 5, 6, 7, 8, 9 and 11 where they support the discussion. Use captions beneath the figures and refer to each figure by number in the corresponding task section.